# Comparaison d'images par grille binaire

## Introduction

La comparaison d'images est une étape cruciale dans les domaines de la vision par ordinateur, de la reconnaissance visuelle et du traitement d'images. Dans ce mini-projet, nous avons développé une application en Python utilisant `Tkinter` et `PIL` permettant de mesurer la similarité entre deux images en utilisant des méthodes optimisées sur une grille de pixels. L'objectif est de détecter rapidement les différences ou ressemblances entre deux images, même en présence de rotations ou variations mineures.

## Méthode de comparaison utilisée

L'application repose sur une approche **par grille binaire**, combinant plusieurs critères pour évaluer la similarité entre deux images. Le processus principal comprend trois étapes :

### 1. Transformation des images en matrices de luminance

Chaque image est convertie en matrice de luminance à partir des valeurs RGB de chaque pixel, et les pixels blancs sont identifiés et ignorés. Cette étape permet de simplifier l'analyse en se concentrant sur les zones avec contenu visuel.

### 2. Réduction et binarisation sur une grille

Les images sont redimensionnées en une grille de taille fixe (`GRID_SIZE`×`GRID_SIZE`). Chaque cellule de la grille est évaluée pour déterminer si elle contient du contenu ou non, en combinant :

- La proportion de pixels non blancs,
- La détection de contours (edges),
- Un seuil adaptatif basé sur la moyenne et la médiane des luminances.

Le résultat est une grille binaire indiquant la présence ou l'absence de contenu dans chaque cellule.

### 3. Comparaison des grilles binaires

Les deux grilles binaires sont comparées selon trois critères :

1. **Similarité de contenu :** proportion de cellules correspondantes entre les deux grilles.
2. **Similarité de forme :** ratio du nombre de cellules avec contenu dans chaque image.
3. **Similarité de position :** comparaison du centre de masse des zones avec contenu.

Une pondération finale combine ces trois critères pour produire un pourcentage de similarité global. L'application teste également plusieurs transformations de l'image (rotations de 90°, 180°, 270°) pour trouver la meilleure correspondance.

## Fonctions principales

- `img_to_luminance_matrix_fast` : Convertit une image en matrice de luminance et identifie les pixels blancs.
- `downscale_to_grid_fast` : Réduit l'image en une grille et détecte le contenu dans chaque cellule.
- `grid_to_binary_fast` : Transforme la grille en binaire selon un seuil adaptatif.
- `compare_binary_grids_fast` : Compare deux grilles binaires et calcule la similarité en combinant contenu, forme et position.
- `rotate90_bin, rotate180_bin, rotate270_bin` : Permettent d'effectuer des rotations sur les grilles pour tester différentes orientations.

## Références utilisées


- Image Processing with Pillow – Python : Explication sur la conversion d'images en matrices et manipulation pixel par pixel.
- Contours et détection de forme en Python : Détection de contours et importance pour la comparaison de contenu.
- Comparaison simple de deux images en Python : Méthode pour comparer des images et mesurer la similarité globale.


## Conclusion

Ce mini-projet démontre une approche simple mais efficace pour comparer des images en utilisant une grille binaire et une pondération combinant contenu, forme et position. L'utilisation de la grille réduit la complexité de calcul tout en restant suffisamment précise. L'application est robuste aux rotations et variations mineures, et constitue une base pour des méthodes plus avancées de reconnaissance et comparaison d'images.


Réalisé par : **Kadi Assirem** & **Akssyl Makhlouf**

Encadré par : **M. Goutal**